

实验一:求最大公约数

- 实验内容:求两个自然数 和 的最大公约数。
- 实验目的：
 - 复习数据结构课程的相关知识，实现课程间的平滑过渡；
 - 掌握并应用算法的数学分析和后验分析方法；
 - 理解:不同的算法能够解决相同的问题，这些算法的解题思路不同，复杂程度不同，解题效率也不同。
- 实验要求
- 设计并实现四个版本的求最大公约数算法；对所设计的算法采用大 符号进行时间复杂性分析；上机实现算法，并用计时法测算算法的运行时间；通过分析对比，得出自己的结论。
- 实验过程及结果

方法 1
辗转相除法
<pre>1. #include<stdio.h> 2. 3. int main(void) 4. { 5. /*****Begin*****/ 6. //辗转相除法求最大公因数 7. int n, m; 8. scanf("%d %d", &n, &m); 9. int tmp1 = n, tmp2 = m; 10. int t = n % m; 11. while(t) //t 为 0 结束执行 12. { 13. n = m; 14. m = t; 15. t = n % m; 16. } 17. int cM = (tmp1 * tmp2) / m; 18. printf("最大公因数: %d\n 最小公倍数: %d\n", m, cM); 19. 20. /*****End*****/ 21. return 0;</pre>

22. }
实验结果:
1.测试 1:
2 5
最大公约数是: 1
最小公倍数是: 10
2.测试2:
22 10
最大公约数是:2
最小公倍数是:110
复杂度分析:
$O(\sqrt{n})$, 同时模运算效率较高

方法 2
相减法
<pre>1. #include <stdio.h> 2. 3. int main() 4. { 5. //相减法求最大公约数 6. int n, m; 7. scanf("%d %d", &n, &m); 8. int tmp1 = n, tmp2 = m; 9. while(n != m) 10. { 11. if(n > m) 12. n = n -m; 13. else if(n < m) 14. m = m -n; 15. } 16. //此时 n 和 m 均为最大公约数 17. int cM = (tmp1 * tmp2) / n; 18. printf("最大公约数是:%d\n 最小公倍数是:%d\n", n, cM); 19. 20. return 0; 21. }</pre>
实验结果:
1.测试 1:
2 5
最大公约数是: 1
最小公倍数是: 10

2.测试2:

22 10

最大公约数是:2

最小公倍数是:110

算法复杂度分析:

$O(\sqrt{n})$, 但减法较模运算效率低(运算的次数多), 所以相同的例子会略显逊色。

方法 3

枚举法

```
#include <iostream>
#include <algorithm>

using namespace std;

int main()
{
    //穷举法
    int n, m;
    scanf("%d %d", &n, &m);
    int tmp1 = min(n, m);
    for(int i = tmp1; i >= 1 ; --i)
    {
        if((n % i == 0 && m % i == 0) || i == 1)
        {
            tmp1 = i;
            break;
        }
    }
    //此时 n 和 m 均为最大公约数
    int cM = (n * m) / tmp1;
    printf("最大公约数是:%d\n 最小公倍数是:%d\n", tmp1, cM);

    return 0;
}
```

实验结果同上

算法复杂度分析:

$O(N)$

方法 4

Stein 法

```

#include <iostream>
#include <algorithm>

using namespace std;      //别忘了加这句话!!

//stein 法实现最大公约数
int gcd(int a,int b){
    if(a<b)
    {
        //arrange so that a>b
        int temp = a;
        a = b;
        b=temp;
    }

    if(0==b)//the base case (递归终止条件)
        return a;

    if(a%2==0 && b%2 ==0)//a and b are even
        return 2*gcd(a/2,b/2);

    if ( a%2 == 0)// only a is even
        return gcd(a/2,b);

    if ( b%2==0 )// only b is even
        return gcd(a,b/2);

    return gcd((a+b)/2, (a-b)/2); // a and b are odd
}

int main()
{
    //穷举法
    int n, m;
    scanf("%d %d", &n, &m);
    int gcd1 = gcd(n, m);

    //此时 n 和 m 均为最大公约数
    int cM = (n * m) / gcd1;
    printf("最大公约数是:%d\n最小公倍数是:%d\n", gcd1, cM);

    //
    return 0;
}

```

```
}  
/*  
测试用例:  
3 5  
22 10  
*/
```

实验结果同上

算法复杂度分析:

$O(2^n)$

● 实验心得

1. 对于一个问题要考虑多种解法（算法）。
2. 不同的算法要考虑其时间复杂度（所需要付出的代价）
3. 根据具体需要选择合适的算法。